Tech Science Press

ARTICLE

A Lightweight Four-Credential Authentication and Key Agreement Protocol for AI-Enabled Edge-IIoT: Transcript-Bound Resistance to Replay and Impersonation

Xun Zhang[1], Zhiying Mu[1,*], Wu Wei[1], Zhihui Mu[1], Dejun Mu[1] and Xin Liu[2]

[1]School of Cyberspace Security, Northwestern Polytechnical University, Xi'an 710072, China
[2]School of Artificial Intelligence, Lanzhou University, Lanzhou 730000, China
*Corresponding Author: Zhiying Mu. Email: muzhiying@mail.nwpu.edu.cn

Version October 6, 2026 submitted to Comput Mater Contin

ABSTRACT: Edge intelligence places model inference and data aggregation on edge nodes deployed outside tightly controlled premises. The edge node therefore decides who may query a deployed model and whose measurements may reach its input. We present an authentication and key agreement scheme using a password, smart card, biometric, and Cognitively Evaluable Authentication Factor (CEAF). CEAF is a session-dependent knowledge credential derived from a fresh challenge and checked against edge-side state. The hash–XOR exchange uses nonces and timestamps, while version-bound two-record state tolerates one lost authentication message without re-registration. The ROR analysis separates password and biometric guessing, transcript collisions, forgery, and CEAF-response enumeration. Sensitivity analysis shows how the residual guessing probability changes with the segment count and the admitted-attempt budget. The main protocol provides session-key independence but neither perfect forward secrecy nor key opacity from the edge node. The analytical unit-cost estimate is 0.4588 ms; a host reference implementation measured 0.0226 ms for the summed software roles on an AMD Ryzen 7 7800X3D, excluding communication and I/O. The construction is a pre-inference access-control mechanism for edge-intelligent IIoT, not a model-level defense. In particular, because Q_1 is a transcript-bound hash, exposure of the smart card, password, and biometric permits offline enumeration of the CEAF response from a recorded login; the scheme does not claim offline resistance under that compromise.

KEYWORDS: security guardrails for AI systems; AI-enabled Industrial Internet of Things; edge intelligence; pre-inference access control; authentication and key agreement; automated credential attacks

1 Introduction

The convergence of the Internet of Things and cyber-physical systems has made IIoT networks responsible for time-sensitive monitoring and control. Sending all sensor data to a remote cloud adds delay, bandwidth contention, and exposure of operational data [1]. Edge computing moves computation and storage closer to the source [2,3]. It also hosts learned models for anomaly detection, predictive maintenance, and visual inspection. The "user–edge node–sensor node" architecture therefore describes an edge-intelligent system: the edge node authenticates users, aggregates sensor data, relays traffic, and serves local model queries.

This architectural change also moves the trust boundary toward resource-constrained or physically accessible devices connected by public wireless channels [4,5]. Once the edge node hosts inference, a failed access decision can expose model queries or admit fabricated model inputs. Authentication is therefore the

© 2026 by the Authors. Submitted to Comput Mater Contin for under a Creative Commons Attribution 4.0 International License, which permitted unrestricted use, distribution, and reproduction in any medium, provided the original work is properly cited.

outermost guardrail and must precede model-level defenses. An AKA protocol for this environment must authenticate all three parties, establish a user–sensor key, preserve endpoint anonymity on the channel, and resist replay, modification, impersonation, and man-in-the-middle attacks [6–8]; related lightweight constructions make the same requirements explicit [9,10]. We treat the TEE-protected edge node as a trusted intermediary and state this assumption explicitly in Sections 3.4 and 7.

Static passwords remain vulnerable to low-entropy choice, reuse, leakage, keylogging, and observation even when a card or biometric is added [11–13]; related analyses report the same exposure in deployed authentication systems [14]. Learned password models and generative biometric synthesis increase the rate of offline guesses and presentation attacks [15–17]; phishing and credential stuffing extend the same exposure to large-scale attempts. Work performed on the attacker's own hardware is outside a defender-controlled rate limit, so protocols that rely on static credentials inherit this exposure.

A dynamic credential can reduce replay and single-observation exposure by making the accepted response session dependent [18]. Conventional dynamic-password or OTP designs typically require a token or repeated secret entry; local-verifier variants place additional verification material on the login device. CEAF keeps its verifier at *EN* and asks the user to select one candidate per segment from a fresh challenge. Unlike a fixed indexed PIN, randomized candidate positions do not expose a permanent index-to-response mapping; unlike a token-based OTP, CEAF needs no additional hardware; unlike a full manual dynamic password, it reduces the calculation to one selection per segment. At $s = 3$ this is three selections rather than 21 manual shifts, but randomized display, lighting, training, and observation affect human error and shoulder-surfing risk; no empirical error rate is claimed. The response must still be bound to the current transcript, and live requests remain subject to source/global controls. A recorded Q_1 remains an offline test when the other credentials are exposed.

Existing constructions usually satisfy only part of this objective. Hash-based WSN schemes with changing passwords have low cost but do not directly model an edge intermediary. ECC-based multi-factor schemes strengthen authentication but require point multiplication, while recent edge-specific protocols often retain a static password or support only two parties [19–21]. Other multi-factor and edge-oriented constructions make related trade-offs [22–24]; lightweight hash–XOR and batching designs provide further baselines [25–27]. The design gap is a user-generated response that is dynamic, transcript-bound, and inexpensive for constrained devices. The present construction addresses that gap while treating offline CEAF enumeration after multi-credential compromise as an explicit limitation.

We introduce a Cognitively Evaluable Authentication Factor (CEAF) and build a four-credential AKA scheme around it. A fresh challenge and a memorized, evolving secret determine each response. The login device displays candidates and collects the user's choice; the first-message authenticator binds that response to the transcript. Here, a *credential* means one provisioned secret or evidence item that can be compromised and accounted for separately. Password and CEAF are both knowledge credentials, while the smart card represents possession and the biometric represents inherence; therefore "four-credential" does not assert four statistically independent factor categories. The term describes the four independently provisioned inputs used by the protocol, not a claim that the scheme supplies four classical factor classes.

The construction is deliberately non-learning: no classifier or learned component appears in the protocol. It supplies the access-control layer for an edge-intelligent deployment and is evaluated as a protocol mechanism, not as a detector.

The mechanism controls admission: which principals may open a session, which sensor may inject measurements, and which requests reach local inference. It does not control the model after an authenticated request is admitted. Prompt injection, training-data or model poisoning, adversarial examples, and model

extraction or membership inference are outside the protocol and are discussed in Section 2.3. Section 3.3 records this boundary. The contributions are:

1. A segmented cognitive primitive: a response guess succeeds with probability 26^{-s}, and two consecutive observations recover its state.
2. Edge-side CEAF state and transcript binding, together with source/global ingress limits for live attempts. A recorded Q_1 remains an offline test after exposure of the card, password, and biometric; this limitation is included in the security analysis.
3. Two-record state that tolerates one lost message in the four-message authentication exchange, so that a dropped message costs one round of availability instead of forcing re-registration.
4. An ROR bound that separates credential guessing, transcript collisions, forgery, and CEAF-response enumeration, and records the protocol's negative security results.

2 Related Work

Related AKA schemes fall into two architectural stages. Earlier WSN work places a gateway between the user and sensor. Recent edge-IIoT work replaces it with an edge node whose deployment, resources, and trust assumptions differ. The WSN literature still supplies many relevant lightweight authentication ideas.

2.1 Authentication Schemes in the WSN Stage

Wong et al. [28] introduced dynamic authentication for resource-constrained WSNs. Das [29] identified weaknesses in early designs; Paliwal [30] studied authentication for industrial sensing. Turkanović and Hölbl [21] developed a dynamic-password construction later extended to heterogeneous IoT deployments [31]. Multi-gateway, anonymity-preserving, temporal-credential, and biometric schemes broadened the design space [22,23,32]. This line of work is important for lightweight protocol design, but most schemes assume a conventional gateway and many retain a static or low-entropy first factor.

2.2 Authentication Schemes in the Edge IIoT Stage

Purpose-built edge-IIoT schemes reflect different trade-offs. Cui et al. [26] amortize authentication through batching, but bilinear pairings remain expensive for constrained devices and their semi-trusted edge model differs from ours. Shang et al. [27] use a lightweight hash–XOR construction for edge control systems, but consider two-party authentication without a smart-card-based user factor. Xu et al. [25] combine a password, smart card, and fuzzy biometric extraction with ECC; the design provides broad security coverage at the cost of several point multiplications and still retains a static password. TEE-assisted and privacy-preserving alternatives are also represented by the DAC–Intel SGX construction of Liu et al. [33] and the lightweight three-factor LAPTAS scheme of Far et al. [34]; their problem settings and mechanisms differ from the CEAF transcript binding and version-bound recovery objectives studied here. These schemes motivate a construction that combines a dynamic factor, multi-factor three-party authentication, and hash-dominated device cost. Recent edge-AI security work examines the security applications of edge intelligence [35], privacy-preserving federated learning with an edge server [36], and resource-efficient authentication for digital-twin edge networks [37]. These studies protect AI services, distributed training, or adjacent edge-network deployments; they do not evaluate the CEAF transcript binding and two-record recovery combination proposed here. The numerical tables accordingly retain only schemes with compatible parties and session-key functionality.

The broader edge-systems literature also covers security-aware task offloading in vehicular edge computing [38] and live migration of correlated virtual machines in cloud data centers [39]. Related data and service-security studies include imbalanced streaming-data clustering [40], ADS-B Tree-Chain authentication [41], and Byzantine-robust privacy-preserving cloud–edge federated learning [42]. These works are cited to acknowledge the wider special-issue context. Their primary objectives and evaluation metrics concern offloading, migration, clustering, aviation-message integrity, or federated-model robustness, respectively, so they are not treated as direct credential-level AKA baselines for the present protocol.

2.3 Security of AI Deployments at the Network Edge

A third body of work concerns model security at the edge. Edge-intelligence surveys describe exposed model-serving interfaces [43,44]; prediction APIs enable model extraction and membership inference [45,46], while adversarial-learning taxonomies cover evasion, poisoning, and privacy threats [47–49]. Indirect prompt injection and training-data poisoning are discussed in more recent analyses [50,51].

These attacks assume that a caller can already reach the model. Access control limits who can issue queries but does not protect an authenticated caller. Credential attacks are also increasingly automated: model-assisted password guessing [15,16] and generative biometric synthesis [17] weaken the first three credentials. The present mechanism addresses the admission layer, as specified in Section 3.3.

3 Preliminaries

Hash-length convention. Throughout the protocol specification, $h(m)$ denotes the leftmost 160 bits of SHA-256(m), written $H_{160}(m) = \mathrm{Trunc}_{160}(\mathrm{SHA256}(m))$. The 160-bit profile is used for serialized protocol fields, communication accounting, and static-state accounting. The host benchmark measures the underlying full SHA-256 compression operation; truncation is applied only when a digest is used as a protocol field and therefore does not change the counted hash call. A deployment using full 256-bit fields would require a separate communication and storage calculation.

3.1 Cognitive Binding Primitive

The CEAF lets a user mentally derive a session response from a fresh challenge without storing the CEAF state on the smart card. Its role is cognitive participation and remote factor verification, not encryption. Protocol-message integrity and key establishment remain the responsibility of hashes, nonces, and timestamps. The following construction makes the response space and observation limits explicit rather than treating the cognitive factor as an unspecified human secret.

3.1.1 Challenge derivation

Let Σ be the 52 case-sensitive Latin letters and L the challenge length. Rather than interpret random bytes as letters, the login device samples a 160-bit seed σ and computes

$$RS = \mathrm{Exp}(\sigma) = \big(RS[1], \ldots, RS[L]\big) \in \Sigma^L, \quad \text{(P1)}$$

Exp reads the 20-byte outputs $h(\sigma \,\|\, 1) \,\|\, h(\sigma \,\|\, 2) \,\|\, \cdots$, accepts bytes below $208 = 4 \cdot 52$, and reduces them modulo 52. Under the random-oracle/PRF model, symbols are unbiased and the $L = 21$ output is statistically close to uniform over Σ^L. Two 160-bit hash blocks suffice except with probability 7.96×10^{-6}, giving about 2.000008 hashes in expectation. Only the masked 160-bit seed is transmitted. The expected challenge-expansion calls are included in the LD and EN hash counts reported in Table 4; they are not an additional cost term.

3.1.2 Segmented offset primitive

Fix a segment count $s \geq 1$ and a segment length ℓ with $L = s\ell$, and write $RS = RS^{(1)} \parallel \cdots \parallel RS^{(s)}$ for the partition of the challenge into s consecutive blocks of ℓ characters. The user-memorized secret is an offset vector $\mathbf{N} = (N^{(1)}, \ldots, N^{(s)}) \in \mathbb{Z}_{26}^{s}$ together with a step vector $\mathbf{N}_S \in \mathbb{Z}_{26}^{s}$. The primitive is

$$\mathrm{CT}_{\mathbf{N}}(RS) = \mathrm{CT}_{N^{(1)}}\left(RS^{(1)}\right) \parallel \cdots \parallel \mathrm{CT}_{N^{(s)}}\left(RS^{(s)}\right), \quad \text{(P2)}$$

where CT_n shifts letters by n positions modulo 26 while preserving case [52]. The workflow is:

1. At registration, U chooses $(\mathbf{N}, \mathbf{N}_S)$ and sends them to EN over the secure channel; no copy is stored on SC (Section 4.2).
2. At login, LD displays for each segment the 26 shifted candidates. U selects one per segment, producing DPW.
3. After confirmation, U and EN set $\mathbf{N} \leftarrow \mathbf{N} + \mathbf{N}_S \bmod 26$; $\mathbf{N}_S$ changes only during credential update.

We use $s = 3$, $\ell = 7$, and $L = 21$ as a reference configuration, yielding $26^3 = 17576$ responses. The user makes one candidate selection per segment rather than manually computing 21 modular shifts. In deployment, candidate positions should be randomized independently for each segment, rendered as high-contrast alphabetic glyphs, and cleared after a short bounded timeout; error feedback should not reveal the correct candidate. A fixed indexed layout would reduce the response to a static short code. The $s = 3$ reference configuration limits the interaction to three selections; $s = 4$ is a possible higher-risk setting, whereas $s \geq 5$ should be reserved for workflows with training and sufficient viewing time. Increasing s raises the response space but also increases user selections, so the parameter choice requires usability testing in the target industrial workflow. No empirical human-error rate is claimed here.

Segmenting the challenge serves two purposes. It expands the per-session response space from 26 to 26^s while keeping each user decision limited to one alphabetic column, and it permits the state to evolve independently in each segment. The login device can generate and display all candidates from the public challenge, but it cannot determine which candidate is correct because neither $\mathbf{N}$ nor $\mathbf{N}_S$ is stored locally. The edge node retains those vectors in masked form and evaluates the response only when reconstructing Q_1.

3.1.3 Security of the primitive

Guessing. A response guess succeeds with probability $26^{-s} = 2^{-14.10}$ at $s = 3$. Rejected M_1 messages do not update the target user's credential record, increment a per-user counter, or start a user-specific cooldown before Q_1 verifies; therefore, a plaintext PID_i cannot by itself lock the user out. A deployment may place a stateless challenge cookie or other anti-spoofing admission token before allocating per-source state, then apply source/global ingress token buckets, replay filtering, and concurrency limits that do not use PID_i as a lock key. These are standard layered DDoS controls [53]: they bound processing and live attempts per observation interval but do not impose a lifetime bound or prevent distributed flooding.

Observation. One observed response exposes only that session's $\mathbf{N}$; two consecutive successful observations reveal $\mathbf{N}_S = \mathbf{N}_{t+1} - \mathbf{N}_t$ and all later responses. Thus the CEAF state is not recovered from one observation alone. The one-observation limit is an environmental assumption about the attacker obtaining a usable challenge–response pair (for example, through line of sight or recording), not a cryptographically enforced property. Combining one observation with the card, password, and biometric compromises all four credentials and may enable one adjacent-state attempt through the recovery window; that full-compromise case is not claimed secure.

Enumeration. With the card, password, and biometric, an adversary can test all 26^s offsets against Q_1. This case is outside the general two-corruption experiment, but shows that the CEAF contributes only

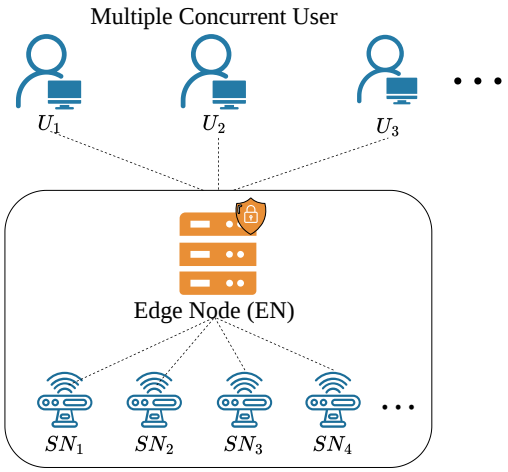


Figure 1: Network model of the edge-collaborative IIoT scheme.

14.10 bits at $s = 3$. Hashing and live rate limits do not change this offline bound; removing it would require a larger or differently designed response space, or an additional uncompromised server-held secret. CEAF binds a human response to a session; it does not provide confidentiality.

3.2 Network Model

Figure 1 shows one edge domain containing multiple users and sensor nodes. A user initiates an authenticated request, a sensor supplies the requested process or environmental data, and the edge node is the sole routing and authentication intermediary. In an edge-intelligent deployment the same node also runs local analytics or model inference over the aggregated measurements, so the decision taken here fixes both who may issue a request and whose data enters the local processing pipeline; the protocol below is agnostic to what that pipeline computes. U and SN register with EN over a protected channel at deployment time; later AKA sessions use the two public wireless links without administrator involvement. Direct U–SN communication is not assumed.

Trust and resources. A TEE protects EN's protocol logic and long-term secrets. This assumption matches an edge gateway deployed as an operator-controlled trust anchor, where the TEE is intended to isolate protocol state from the host OS while the gateway must still authorize traffic and route sensor data. Physical access to the edge hardware is allowed, but extraction of TEE-protected values is outside the main model. Breaking this boundary exposes every stored CEAF state; a deployment that cannot provide this isolation is outside the stated security claim. Moreover, because the main protocol lets EN recover X and Y, the edge node also knows the session key; Section 7 records this trusted-intermediary limitation relative to semi-trusted designs [26]. A semi-trusted intermediary would be expected to relay or verify messages without learning the endpoint key, which would require a different key-establishment design and is not the trust goal adopted here. SN has the tightest computation and storage budget and therefore uses only hashes and XOR. U and EN are assumed to have renewable power and sufficient resources for biometric reproduction, challenge expansion, and state management.

3.3 The Edge Node as a Pre-Inference Control Point

Figure 1 is usually read as a routing diagram. Under edge intelligence it is better read as a control diagram, because two distinct paths converge on EN and both of them terminate at the local model rather than at a data store.

Table 1: Coverage of the authentication guardrail at an AI-enabled edge node.

Attack or exposure	Addressed here	Where
Unauthenticated access to the model-query or inference interface	Y	Sec. 4.4, Q_1
Unauthenticated or spoofed sensor input entering the aggregation path	Y	Secs. 4.2, 4.4
Offline attack using a leaked verifier or recorded transcript	Partly	Secs. 3.1.3, 5.1.1
Exposure of up to three of the four credentials	Partly[a]	Sec. 5.1.1
Replay, modification, impersonation, man-in-the-middle	Y	Sec. 5
Endpoint identity exposure to a channel observer	Y[b]	Sec. 5
Loss of one authentication message (credential desynchronization)	Y[c]	Sec. 4.4.1
Distributed flooding of the edge node	Partly[d]	Sec. 5
Prompt injection against a generative component	N	Sec. 2.3
Training-data or model poisoning after admission	N	Sec. 2.3
Adversarial examples at inference time	N	Sec. 2.3
Extraction or membership inference by an authenticated client	N[e]	Sec. 2.3
Detection of AI-assisted malware, ransomware, or intrusion	N	outside scope
Extraction of TEE-protected edge secrets	N	Sec. 3.2

[a] With (SC, PW, BIO) exposed, a recorded Q_1 permits offline enumeration of the CEAF response; live attempts remain subject to deployment source/global limits, but no offline resistance is claimed. [b] Anonymity from the channel, not from EN. [c] The two-record window absorbs one lost M_1–M_4 message; repeated sensor-side loss invokes authenticated M_{R1}–M_{R2} recovery, while credential update uses a separate bounded authenticated retry. [d] Source/global ingress and concurrency limits reduce processing pressure but do not prevent distributed flooding. [e] The guardrail bounds *who* may query, which reduces the population able to mount such attacks, but gives no protection once a client is authenticated.

On the request path, U authenticates to EN and, only after the four-credential transcript check succeeds, obtains a session in which it may issue a query against the sensing service and, in a deployment that hosts analytics, against the inference service exposed by the same node. An unauthorized request at this position is not merely an unauthorized read: it is an unauthorized query against a deployed model, and repeated queries from that position are the standard precondition for extraction and membership-inference attacks [45,46]. On the ingestion path, SN authenticates to EN under TC_j before its measurements are accepted into the aggregation that feeds local processing. An unauthorized sensor at this position is a channel through which fabricated observations reach the model's input, which is the deployment-time analogue of data poisoning [49,51]. The protocol is agnostic to what the local pipeline computes; what it fixes is the pair of admission decisions in front of it.

The assets guarded are therefore the model-query interface, the local inference service, and the sensor-input path, together with the session key that protects the resulting user–sensor exchange. Table 1 states which attacks against those assets this paper claims to address and which it deliberately leaves to the model-level layer, so that the security claims of Sections 5 and 6 are read against a fixed boundary.

3.4 Threat Model

We use the Dolev–Yao model [12,33,54]. The adversary A may:

- control both public channels by eavesdropping, replaying, modifying, dropping, and injecting protocol messages, and also act as a registered malicious user;
- compromise at most two of the password, smart card, biometric, and CEAF credentials in the general model; we additionally analyze simultaneous exposure of the password, smart card, and biometric, in which case a recorded Q_1 permits offline CEAF enumeration and live attempts remain rate controlled. The latter is a deliberately separate stress test: it diagnoses the residual CEAF entropy after three-credential exposure and does not replace the general two-corruption ROR experiment;

Table 2: Notation definitions.

Symbol	Meaning
$U,\ EN,\ SN$	User, edge node, sensor node
$LD,\ SC$	User device and smart card
$SA,\ \mathrm{A}$	System administrator, adversary
$ID_i, ID_{SC}, ID_{SN}, ID_{EN}$	Unique identities of U, SC, SN, EN
PID_i, PID_j	Persistent pseudo-identities of U and SN
PID_{SN}	One-time pseudo-identity of SN
$PW_i,\ SK$	User static password and session key
$TS,\ \Delta T,\ \sigma$	Timestamp, acceptance window, and 160-bit challenge seed
$r_R,\ C_R^{cur},\ C_R^{prev}$	Resynchronization nonce and masked sensor-version identifiers
$RS,\ \mathrm{Exp}(\cdot)$	Challenge $RS = \mathrm{Exp}(\sigma) \in \Sigma^L$ and its expansion function
$s,\ \ell,\ L$	Segment count, segment length, challenge length $L = s\ell$
$\mathbf{N},\ \mathbf{N}_S$	CEAF offset vector and step vector in $\mathbb{Z}_{26}^s$
DPW	One-time CEAF response $\mathrm{CT}_{\mathbf{N}}(RS)$
$\mathrm{Gen}(\cdot)/\mathrm{Rep}(\cdot)$	Generation/reproduction algorithm of the fuzzy extractor
$\mathrm{CT}_{\mathbf{N}}(\cdot)$	Segmented cognitive-binding primitive with offset vector $\mathbf{N}$
$h(\cdot),\ \|,\ \oplus$	Hash function, concatenation, XOR
$\mathrm{enc}_s, \mathrm{dec}_s$	Fixed-length encoding of $\mathbb{Z}_{26}^s$ and its inverse on the encoding image
$T_H,\ T_F$	Analytical unit costs of one hash and one fuzzy-extractor operation; values are illustrative

- capture a sensor and read all of its storage; security then concerns lateral compromise of uncaptured nodes, while the captured node itself requires revocation or re-registration;
- access edge hardware but not extract TEE-protected secrets; and
- observe the displayed challenge and selected CEAF response for at most one successful login. This observation does not by itself expose the password, card state, or biometric; combining it with all three is full credential compromise and is outside the security claim. Two consecutive CEAF observations recover $(\mathbf{N}, \mathbf{N}_S)$ and are likewise excluded; this is an environmental observation bound, not a cryptographic guarantee; and
- automate password guessing or biometric synthesis. Automation changes the rate and per-attempt success probability, not the set of values obtainable by A. Offline work against a password verifier or biometric template is assumed to proceed at the adversary's hardware speed. The one-successful-login limit on CEAF observation is a modeling assumption and is revisited in Section 7.

The adversary is correspondingly *not* assumed to attack the model behind the guardrail. Adversarial examples, poisoning of the locally hosted model, prompt injection against a generative component, and extraction or membership inference mounted from a session that has already been admitted are excluded by assumption rather than defended against; Table 1 records the division and Section 2.3 identifies the layer to which those attacks belong. A registered malicious user may deviate arbitrarily in the protocol, but no claim is made about what it does with the inference service once a session is established.

Corruption of R_i, TC_i, TC_j, or R_{EN} makes a main-protocol session non-fresh because these values reconstruct transcript nonces. Hash inversion, second-preimage construction, collision finding, nonce prediction, and compromise of the secure registration channel are assumed infeasible. All claims in Section 5 are scoped to these freshness and compromise conditions.

3.5 Notation

The notation used throughout this paper is listed in Table 2.

4 The Proposed Scheme

The scheme has initialization, registration, login, AKA, and update phases. Verification is divided between two locations: *LD* checks the password, card, and biometric, while *EN* checks CEAF through the first protocol authenticator. The transcript binding prevents a verifier from being stored on the card, but a recorded Q_1 can still be tested offline after the other three credentials are exposed. Apart from fuzzy reproduction and challenge expansion, the protocol uses hashes, XOR, nonces, and timestamps. Rotating pseudo-identities and CEAF state are maintained as version-bound two-record windows to tolerate one lost authentication message. Protocol displays are numbered $(P1), (P2), \ldots$ for implementation reference; the analytical security bounds use ordinary equation numbers (1)–(3).

4.1 System Initialization Phase

Before deployment, the system administrator *SA* provisions each device as follows:

1. Install the primitive set $\{\oplus, h(\cdot), \|\}$ on every *LD*, *EN*, and *SN*.
2. Install $\mathrm{Exp}(\cdot)$ from Section 3.1.1 on every *LD* and *EN*; install $\mathrm{CT}_{\mathbf{N}}(\cdot)$ on every *EN* and in the *LD* display routine, and install $\{\mathrm{Gen}(\cdot), \mathrm{Rep}(\cdot)\}$ on every *LD*.
3. Fix the CEAF parameters (s, ℓ) and timestamp-acceptance window ΔT system-wide; this paper uses $(3, 7)$. Let $\langle\cdot\rangle = \mathrm{enc}_s(\cdot)$ denote the fixed-length 160-bit encoding of a vector in $\mathbb{Z}_{26}^s$, and let dec_s be its inverse on the encoding image. Thus dec_s is string decoding, not group inversion. Source/global ingress budgets are deployment parameters and are not stored in Λ_i.
4. Assign a unique identity ID_{SC} to each smart card, ID_{SN} to each sensor node, and ID_{EN} to the edge node. Each edge domain samples one master secret R_{EN} and seals it inside the TEE before any registration.

4.2 Registration Phase

Registration uses a secure channel and binds each user to a unique smart card and each sensor to the edge domain. Long-term credentials are stored in masked form, and R_{EN} remains inside the TEE. The user sends the CEAF vectors to *EN* only during registration and erases the working copies on *LD*; this placement prevents the card or login device from becoming a CEAF verification oracle.

4.2.1 User Registration

1. *U* supplies its identity, password, biometric, and CEAF vectors. *LD* reads ID_{SC}, samples R_i, obtains TS_1, and computes

$$\begin{aligned} (R_U, P_U) &= \mathrm{Gen}(BIO_i), \\ UID_i &= h(ID_i \,\|\, PW_i), \\ HPW &= h(ID_i \,\|\, PW_i \,\|\, ID_{SC} \,\|\, R_U \,\|\, R_i), \end{aligned} \tag{P3}$$

   then sends the registration tuple

$$\{UID_i, ID_{SC}, HPW, R_i, \langle\mathbf{N}\rangle, \langle\mathbf{N}_S\rangle, TS_1\} \tag{P4}$$

   to *EN* over the secure channel and erases the CEAF vectors.
2. After checking TS_1, *EN* uses its TEE-resident domain secret R_{EN}, obtains TS_2, and computes

$$\begin{aligned} TC_i &= h(R_{EN} \,\|\, R_i), \\ PID_i &= h(UID_i \,\|\, ID_{SC} \,\|\, R_i), \\ PTC_i &= TC_i \oplus h(R_i \,\|\, UID_i \,\|\, HPW), \\ RPW &= h(TC_i \,\|\, R_i), \end{aligned} \tag{P5}$$

returns $\{PID_i, RPW, PTC_i, TS_2\}$ to LD. EN then forms the masked credential state

$$\begin{aligned} PK_i &= R_i \oplus h(R_{EN} \,\|\, ID_{EN}), \\ PN_{EN} &= \langle \mathbf{N} \rangle \oplus h(TC_i \,\|\, R_{EN}), \\ PNS_{EN} &= \langle \mathbf{N}_S \rangle \oplus h(TC_i \,\|\, ID_{EN} \,\|\, R_{EN}), \end{aligned} \tag{P6}$$

and stores the two-record state

$$\Lambda_i = \{PID_i^{cur}, PID_i^{prev}, PK_i, PN_{EN}, PNS_{EN}\}, \tag{P7}$$

with $PID_i^{cur} = PID_i^{prev} = PID_i$. Because PK_i is masked by R_{EN} rather than PID_i, only the rotating identifier is duplicated. The CEAF vectors remain only at EN; invalid authentication attempts do not modify this record.

3. After checking TS_2, LD computes

$$PR_i = R_i \oplus h(UID_i \,\|\, ID_{SC} \,\|\, R_U) \tag{P8}$$

and stores the five fields PID_i, RPW, PTC_i, PR_i, and P_U, then erases its working secrets. The provisioned public identifier ID_{SC} is not counted as cryptographic state, and SC stores no CEAF representation.

4.2.2 Sensor-Node Registration

Sensor registration proceeds as follows.

1. SN takes its own identity ID_{SN}, samples R_j, computes $PID_j = h(ID_{SN} \| R_j)$ and $TC_j = h(R_j \| ID_{SN} \| ID_{EN})$, obtains TS_1, and sends $\{ID_{SN}, R_j, TS_1\}$ to EN. It retains PID_j until the returned state is installed.
2. Upon receipt, EN first checks the freshness of TS_1; if it has expired, EN aborts. Otherwise EN obtains TS_2 and computes

$$\begin{aligned} PID_j &= h(ID_{SN} \,\|\, R_j), \\ TC_j &= h(R_j \,\|\, ID_{SN} \,\|\, ID_{EN}), \\ PK_j &= R_j \oplus h(PID_j \,\|\, ID_{EN}), \\ PTC_j &= TC_j \oplus PID_j, \end{aligned} \tag{P9}$$

returns $\{PTC_j, TS_2\}$ to SN, and stores the two-record state $\Lambda_j = \{PID_j^{cur}, PID_j^{prev}, PK_j\}$ indexed by ID_{SN}, with $PID_j^{cur} = PID_j^{prev} = PID_j$.

3. Upon receipt, SN first checks the freshness of TS_2; if it has expired, SN aborts. Otherwise SN stores $\{PID_j^{cur}, PID_j^{prev}, PTC_j^{cur}, PTC_j^{prev}\}$, with both PID entries initialized to PID_j and both PTC entries initialized to PTC_j; it then erases R_j and TC_j.

4.3 User Login Phase

The login phase separates local screening from remote authentication. LD first rejects an incorrect password, card, or biometric without creating network traffic. It then collects only the CEAF response; EN verifies that response through M_1. This order limits unnecessary edge and sensor work while preserving the requirement that the fourth credential be checked against edge-only state.

(1) Verification of the first three credentials (static password, smart card, biometric). U inserts SC into LD, inputs $\{ID_i, PW_i\}$, and provides biometric BIO_i. LD reads the smart-card state and computes

$$\begin{aligned} UID_i &= h(ID_i \,\|\, PW_i), \\ R_U &= \mathrm{Rep}(P_U, BIO_i), \\ R_i &= PR_i \oplus h(UID_i \,\|\, ID_{SC} \,\|\, R_U), \\ HPW &= h(ID_i \,\|\, PW_i \,\|\, ID_{SC} \,\|\, R_U \,\|\, R_i), \\ TC_i &= PTC_i \oplus h(R_i \,\|\, UID_i \,\|\, HPW), \\ RPW^* &= h(TC_i \,\|\, R_i). \end{aligned} \tag{P10}$$

If $RPW^* \neq RPW$, *LD* rejects; otherwise the first three credentials pass.

(2) CEAF collection. U selects the target sensor ID_{SN} and verifies the target shown by *LD*. *LD* then samples σ, displays the candidate columns for $RS = \mathrm{Exp}(\sigma)$, and receives DPW. It retains (σ, RS, DPW) only through M_1 construction and does not verify DPW. U advances the offset only after session confirmation.

4.4 Authentication and Key Agreement Phase

U, *EN*, and *SN* authenticate, establish SK for U and *SN*, and rotate credentials. M_1 binds the four-credential login and target sensor, M_2 carries the edge contribution to the selected sensor, M_3 returns the sensor contribution, and M_4 confirms both nonces to the user. Because *EN* is trusted, it learns X, Y, and therefore SK (A15); it verifies the CEAF in step (2). The timestamps provide freshness, whereas the Q values bind the reconstructed fields and prevent an adversary from changing a timestamp or masked nonce independently.

All replay and concurrency checks are atomic. Within ΔT, *EN* caches (PID_i, Q_1, TS_1) for M_1, and *SN* caches (PID_j, Q_2, TS_2) for M_2. For M_3 and M_4, *EN* and *LD* cache (Q_3, TS_3) and (Q_4, TS_4), respectively. A duplicate is rejected before any state transition. *LD* retains one outstanding local context; *EN* and *SN* allow at most one accepted in-flight session for the relevant user and sensor records. M_3 must match the sole pending sensor context at *EN*, and M_4 must match the outstanding context at *LD*. Reservations and replay entries expire after ΔT. These bounded caches and locks are required protocol metadata but are excluded from the static cryptographic-storage comparison in Section 6.

(1) *LD* samples a fresh random number X, obtains a timestamp TS_1, and computes

$$\begin{aligned} PKS_1 &= X \oplus h(R_i \,\|\, TC_i \,\|\, TS_1), \\ PRS &= \sigma \oplus h(X \,\|\, TC_i \,\|\, TS_1), \\ PID_{SN} &= ID_{SN} \oplus h(X \,\|\, TC_i), \\ Q_1 &= h(X \,\|\, ID_{SN} \,\|\, PID_i \,\|\, RS \,\|\, DPW \,\|\, TS_1). \end{aligned} \tag{P11}$$

LD sends

$$M_1 = \{PID_i, PKS_1, PRS, PID_{SN}, Q_1, TS_1\} \tag{P12}$$

to *EN* and erases DPW. Q_1 binds the untransmitted response, while PRS masks the challenge seed.

(2) Upon receipt, *EN* checks the freshness of TS_1 and the M_1 replay key; if either check fails, *EN* aborts. Otherwise *EN* first searches for a state Λ_i whose PID_i^{cur} equals the received PID_i and records $v_i = cur$. If no current value matches, it searches the previous values and records $v_i = prev$. If neither value matches or the record already has an in-flight session, *EN* aborts. Source/global ingress and concurrency controls are applied independently of PID_i; they may reject traffic before this step but never create a user-specific lock. *EN* then recovers

$$\begin{aligned} R_i &= PK_i \oplus h(R_{EN} \,\|\, ID_{EN}), \\ TC_i &= h(R_{EN} \,\|\, R_i), \\ X &= PKS_1 \oplus h(R_i \,\|\, TC_i \,\|\, TS_1), \\ \sigma &= PRS \oplus h(X \,\|\, TC_i \,\|\, TS_1), \quad RS = \mathrm{Exp}(\sigma), \\ ID_{SN} &= PID_{SN} \oplus h(X \,\|\, TC_i), \\ \mathbf{N} &= \mathrm{dec}_s\big(PN_{EN} \oplus h(TC_i \,\|\, R_{EN})\big), \\ \mathbf{N}_S &= \mathrm{dec}_s\big(PNS_{EN} \oplus h(TC_i \,\|\, ID_{EN} \,\|\, R_{EN})\big), \\ Q_1^* &= h(X \,\|\, ID_{SN} \,\|\, PID_i \,\|\, RS \,\|\, \mathrm{CT}_{\mathbf{N}}(RS) \,\|\, TS_1). \end{aligned} \tag{P13}$$

If Q_1 fails and $v_i = prev$, EN retries once with $\mathbf{N}^- = \mathbf{N} - \mathbf{N}_S \bmod 26$; it never tries $\mathbf{N}^-$ for PID_i^{cur}. A match adopts the preceding offset. Any other mismatch aborts without changing Λ_i or any user-specific availability state. After a valid Q_1, EN atomically inserts the M_1 replay key and reserves the user record. Acceptance jointly verifies the four credentials. EN then retrieves Λ_j; if its sensor record is busy, EN releases the user reservation and aborts. Otherwise it reserves that record, uses PID_j^{cur}, obtains TS_2, and computes

$$\begin{aligned} R_j &= PK_j \oplus h(PID_j \,\|\, ID_{EN}), \\ TC_j &= h(R_j \,\|\, ID_{SN} \,\|\, ID_{EN}), \\ PKS_2 &= X \oplus h(TC_j \,\|\, TS_2), \\ Q_2 &= h(X \,\|\, PID_j \,\|\, TS_2), \end{aligned} \tag{P14}$$

and sends $M_2 = \{PID_j, PKS_2, Q_2, TS_2\}$ to the SN designated by ID_{SN}.

(3) After checking TS_2, the M_2 replay key, and the absence of another in-flight session, SN first matches the received PID_j against the stored current/previous pair. If neither identifier matches, it invokes the authenticated resynchronization of Section 4.4.2 and does not process M_2. Otherwise, with $v_j \in \{cur, prev\}$ denoting the matching version, SN recovers $TC_j = PTC_j^{v_j} \oplus PID_j$ and X, then verifies Q_2. If the check fails, it aborts without changing state. On success it atomically inserts the replay key, reserves the record, obtains TS_3, samples Y, derives $SK = h(X \,\|\, Y)$, and computes

$$\begin{aligned} PKS_3 &= Y \oplus h(TC_j \,\|\, X \,\|\, TS_3), \\ Q_3 &= h(X \,\|\, Y \,\|\, TS_3), \end{aligned} \tag{P15}$$

then sends

$$M_3 = \{PKS_3, Q_3, TS_3\} \tag{P16}$$

and derives $PID_j^* = PID_j \oplus h(Y \,\|\, TS_3)$ and $PTC_j^* = TC_j \oplus PID_j^*$. It atomically shifts the sensor state to $(PID_j^*, PID_j, PTC_j^*, PTC_j)$, then releases its reservation while retaining the replay entry for ΔT.

(4) After checking TS_3, matching M_3 to the pending sensor context, and rejecting a cached M_3 key, EN recovers Y and verifies Q_3. It then atomically inserts the replay key, obtains TS_4, and computes

$$\begin{aligned} PKS_4 &= Y \oplus h(R_i \,\|\, TC_i \,\|\, X \,\|\, TS_4), \\ Q_4 &= h(X \,\|\, Y \,\|\, TS_4), \end{aligned} \tag{P17}$$

and sends $M_4 = \{PKS_4, Q_4, TS_4\}$ to the LD held by U. EN then advances both credential states, keeping the value it has just used as the previous entry:

$$\begin{aligned} PID_j^* &= PID_j \oplus h(Y \,\|\, TS_3), \quad PK_j^* = R_j \oplus h(PID_j^* \,\|\, ID_{EN}), \\ PID_i^* &= PID_i \oplus h(X \,\|\, TS_4), \quad \mathbf{N}^* = \mathbf{N} + \mathbf{N}_S \bmod 26, \end{aligned} \tag{P18}$$

$$\begin{aligned} PN_{EN}^* &= \langle \mathbf{N}^* \rangle \oplus h(TC_i \,\|\, R_{EN}), \\ \{PID_j^{prev}, PID_j^{cur}, PK_j\} &\leftarrow \{PID_j, PID_j^*, PK_j^*\}, \\ \{PID_i^{prev}, PID_i^{cur}, PN_{EN}\} &\leftarrow \{PID_i, PID_i^*, PN_{EN}^*\}. \end{aligned} \tag{P19}$$

Here PID_i and PID_j are the values used in this round; PK_i and PNS_{EN} remain unchanged. After the atomic commit, EN releases both in-flight reservations while retaining the replay entries until they expire.

(5) After checking TS_4, matching the sole outstanding context, and rejecting a cached M_4 key, LD recovers Y, derives SK, and verifies Q_4. It then atomically caches the key, stores $PID_i^* = PID_i \oplus h(X \,\|\, TS_4)$, consumes the outstanding context, and signals success, after which U advances $\mathbf{N}$. If M_4 is lost, U retains the old state and step (2) absorbs the next login.

4.4.1 Two-record state and the desynchronization invariant

SN commits when it sends M_3, EN when it sends M_4, and U after verifying M_4. A single lost M_3 or M_4 therefore leaves adjacent principals at most one round apart, which the two-record window absorbs. If repeated loss moves SN beyond the identifiers retained by EN, the normal exchange stops and the authenticated recovery below is invoked; no unauthenticated rollback is allowed. The invariant protects state consistency, not availability under sustained loss: recovery still depends on delivery of the authenticated resynchronization exchange.

Each rotating value is therefore held as a version-bound $(cur, prev)$ pair:

$$\begin{aligned} EN &: (PID_i^{cur}, \mathbf{N}),\ (PID_i^{prev}, \mathbf{N} - \mathbf{N}_S),\ (PID_j^{cur}, PID_j^{prev}), \\ SN &: (PID_j^{cur}, PID_j^{prev}, PTC_j^{cur}, PTC_j^{prev}). \end{aligned} \tag{P20}$$

$\mathbf{N}^-$ is recomputed from $(\mathbf{N}, \mathbf{N}_S)$, avoiding another stored copy. A current PID_i is verified only with $\mathbf{N}$, whereas $\mathbf{N}^-$ is available only after a previous-PID_i match. Thus identifiers and CEAF offsets cannot be mixed across versions: a current identifier is never tested with the previous offset, and a previous identifier is never promoted to the current version without the adjacent-state commit. A previous-value match is accepted only for the immediately adjacent state and is followed by the normal commit rule. Together with the replay cache and single-session reservation, this one-round window recovers one lost M_1–M_4 without accepting a duplicate transcript. Repeated sensor-side loss is handled only by the authenticated resynchronization below; credential update remains separate.

4.4.2 Authenticated sensor resynchronization

If SN receives an M_2 whose PID_j matches neither stored identifier, it derives $TC_j = PTC_j^{cur} \oplus PID_j^{cur}$, samples fresh (r_R, TS_R), and computes

$$\begin{aligned} C_R^{cur} &= PID_j^{cur} \oplus h(TC_j \,\|\, r_R \,\|\, 1), \\ C_R^{prev} &= PID_j^{prev} \oplus h(TC_j \,\|\, r_R \,\|\, 2), \\ Q_R^{(1)} &= h(TC_j \,\|\, C_R^{cur} \,\|\, C_R^{prev} \,\|\, r_R \,\|\, TS_R \,\|\, \mathrm{r1}), \end{aligned} \tag{P21}$$

It sends $M_{R1} = \{C_R^{cur}, C_R^{prev}, r_R, Q_R^{(1)}, TS_R\}$ and waits without changing state. EN recovers R_j from (PID_j^{cur}, PK_j), derives TC_j, checks freshness, replay, and $Q_R^{(1)}$, then cancels the stale M_2 context and releases its reservations. It recomputes $PK_j^* = R_j \oplus h(C_R^{cur} \,\|\, ID_{EN})$, installs $(PID_j^{prev}, PID_j^{cur}, PK_j) = (C_R^{prev}, C_R^{cur}, PK_j^*)$, and returns

$$Q_R^{(2)} = h(TC_j \,\|\, C_R^{cur} \,\|\, C_R^{prev} \,\|\, r_R \,\|\, TS_R \,\|\, \mathrm{r2}), \qquad M_{R2} = \{r_R, Q_R^{(2)}, TS_R\}. \tag{P22}$$

SN accepts M_{R2} only for its outstanding r_R and a valid $Q_R^{(2)}$. It then installs the four recovered fields $(C_R^{cur}, C_R^{prev}, TC_j \oplus C_R^{cur}, TC_j \oplus C_R^{prev})$ and clears the request. If the acknowledgement is lost, it retransmits the same M_{R1} within ΔT and EN returns the cached acknowledgement without another state transition. This idempotent exchange repairs repeated sensor-side loss when the two endpoints can still exchange authenticated messages; it is excluded from the normal-session cost and does not claim availability against distributed flooding or a permanently partitioned link.

4.5 Credential Update Phase

Password and biometric updates are card-local because they only change the masks used to recover R_i and TC_i. Changing the edge-held CEAF vectors requires the protected exchange below. The update is executed only after a complete AKA run so that LD and EN can protect the new vectors with a fresh update key rather than with the user's low-entropy credentials. The replay-cache and single-pending-context

rules of Section 4.4 also apply to M_5 and M_6 within ΔT. The update replay keys are (Q_5, TS_5) at EN and (Q_6, TS_6) at LD. LD keeps the pending update until a valid M_6 is received and retransmits the same M_5 after a timeout; EN caches the accepted update and its M_6 for this retry window and resends it without a second state transition. Thus, one lost M_6 is recovered by an authenticated retry rather than by re-registration.

1. LD completes Sections 4.3–4.4; U and EN set $SK_{UE} = h(TC_i \| X \| Y)$.
2. U selects any subset of a new password, biometric, CEAF vector, and CEAF step vector. An unchanged starred value equals its current value. For a password-only update, $(R_U^*, P_U^*) = (R_U, P_U)$; for a biometric-only update, $PW_i^* = PW_i$. If only one CEAF vector changes, the other starred vector equals its current value.
3. If the biometric changes, LD obtains $(R_U^*, P_U^*) = \mathrm{Gen}(BIO_i^*)$; otherwise it reuses (R_U, P_U). It then recomputes the card-local values

$$\begin{aligned} UID_i^* &= h(ID_i \| PW_i^*), \\ HPW^* &= h(ID_i \| PW_i^* \| ID_{SC} \| R_U^* \| R_i), \\ PTC_i^* &= TC_i \oplus h(R_i \| UID_i^* \| HPW^*), \\ PR_i^* &= R_i \oplus h(UID_i^* \| ID_{SC} \| R_U^*), \end{aligned} \tag{P23}$$

   If neither CEAF vector changes, LD atomically commits the five card fields, using PTC_i^*, PR_i^*, and P_U^*, and erases the working values. The update then ends without contacting EN. If either CEAF vector changes, LD obtains TS_5 and computes

$$\begin{aligned} C_1 &= \langle \mathbf{N}^* \rangle \oplus h(SK_{UE} \| 1 \| TS_5), \\ C_2 &= \langle \mathbf{N}_S^* \rangle \oplus h(SK_{UE} \| 2 \| TS_5), \\ Q_5 &= h(SK_{UE} \| \langle \mathbf{N}^* \rangle \| \langle \mathbf{N}_S^* \rangle \| TS_5), \end{aligned} \tag{P24}$$

   sending $M_5 = \{C_1, C_2, Q_5, TS_5\}$ to EN.
4. Upon receiving M_5, EN first checks its pending-update cache. If (Q_5, TS_5) matches an accepted update still within the retry window, it retransmits the cached M_6 and makes no state change. Otherwise, after checking TS_5 and the replay key, EN recovers

$$\begin{aligned} \langle \mathbf{N}^* \rangle &= C_1 \oplus h(SK_{UE} \| 1 \| TS_5), \\ \langle \mathbf{N}_S^* \rangle &= C_2 \oplus h(SK_{UE} \| 2 \| TS_5). \end{aligned} \tag{P25}$$

   It verifies Q_5, obtains TS_6, and computes

$$\begin{aligned} PN_{EN}^* &= \langle \mathbf{N}^* \rangle \oplus h(TC_i \| R_{EN}), \\ PNS_{EN}^* &= \langle \mathbf{N}_S^* \rangle \oplus h(TC_i \| ID_{EN} \| R_{EN}), \\ Q_6 &= h(SK_{UE} \| \langle \mathbf{N}^* \rangle \| \langle \mathbf{N}_S^* \rangle \| TS_6). \end{aligned} \tag{P26}$$

   EN atomically stores the two starred masks, caches the accepted update and M_6, and returns $M_6 = \{Q_6, TS_6\}$. It never applies an accepted update twice, and no failed M_1 attempt can alter this update state.
5. After checking TS_6 and verifying M_6 against both starred CEAF vectors, LD commits any new card values and U memorizes the new CEAF pair. Until this check succeeds, the old local values are retained. If M_6 is lost, LD retransmits the unchanged M_5 within ΔT; the cached response above is returned and the update is committed only once. If all retries expire, LD reports an incomplete update and retains its old local values; loss-atomicity is not claimed outside this bounded retry window.

4.6 Implementation-Oriented Message Summary

The main exchange can be implemented from the following compact summary; a failed verification leaves the corresponding state unchanged. M_1 ($LD \rightarrow EN$) binds the four credentials, challenge, target sensor, and nonce in Q_1; EN verifies CEAF and reserves the user record. M_2 ($EN \rightarrow SN$) carries the masked edge contribution and Q_2; SN verifies the sensor context and derives its session contribution. M_3 ($SN \rightarrow EN$) returns the sensor contribution and Q_3; SN shifts its two-record state after sending, while EN verifies the pending context. M_4 ($EN \rightarrow LD$) confirms both nonces with Q_4; LD verifies the session and commits its local identifier, while EN commits the adjacent records. Optional M_5 ($LD \rightarrow EN$) and M_6 ($EN \rightarrow LD$) update the CEAF vectors under SK_{UE}; the cached M_6 recovers one lost update acknowledgement without a second state transition.

5 Security Analysis

The Real-or-Random (ROR) reduction bounds transcript and credential-guessing attacks, and the informal audit connects those results to Table 3. The claims are conditioned on the threat model and freshness rules of Section 3.4; in particular, the main protocol claims neither PFS nor key opacity from EN.

5.1 Formal Security Analysis

5.1.1 Reduction in the Real-or-Random Model

Protocol instances answer Execute, Send, Reveal, Hash, CorruptSC, CorruptPW, CorruptBIO, and CorruptCEAF queries, followed by one Test on a fresh accepted instance. Execute exposes honest transcripts, Send models active delivery and modification, and Reveal returns an accepted session key. The general experiment permits at most two credential corruptions for one user and represents the protocol's ordinary partial-compromise claim. A separate three-credential experiment exposes (SC, PW, BIO) together but not the CEAF state; because Q_1 is public in a recorded transcript, the experiment includes offline enumeration of the 26^s possible responses, while live impersonation attempts remain subject to the edge verifier. This stress test is intentionally separated because it measures the residual CEAF entropy after the other credentials are lost; it complements, rather than replaces, the general ROR experiment. Exposure of all four credentials, reveal of the tested key, or corruption of a nonce-recovery value makes the tested session non-fresh. With h a random oracle, $\mathrm{Adv}^{\mathrm{ror}}_{\Pi}(\mathrm{A}) = |2\Pr[\mathrm{Succ}] - 1|$.

Let q_h, q_s, q_n count hash queries, active sends, and sampled nonces, with lengths l_h, l_n. Let q_e count offline CEAF tests against recorded transcripts and q_c count live CEAF trials admitted by the deployment's source/global policy and reaching the CEAF verifier. Each offline or live CEAF test consumes at least one random-oracle query, so the disjoint total $q_{\mathrm{CEAF}} = q_e + q_c$ satisfies $q_{\mathrm{CEAF}} \le q_h$ and $q_c \le q_s$. Under Zipf parameters (C_z, α) [55] and biometric min-entropy l_b, the remaining password and biometric guessing probability satisfies

$$p_{\mathrm{cred}}(q_g) \le \min\left\{1,\ C_z \sum_{r=1}^{q_g} r^{-\alpha} + \frac{q_g}{2^{l_b}}\right\} \tag{1}$$

where q_g counts password and biometric guesses, and $C_z r^{-\alpha}$ is the Zipf probability assigned to rank r. The deployment's source/global admission policy bounds q_c per observation interval; this bound is not a per-user lock and is not assumed to hold for the entire credential lifetime.

The reduction begins with the real attack game G_0. Game G_1 aborts on any hash-output or nonce collision; the birthday terms below bound the statistical difference from G_0. Game G_2 additionally aborts

when an active send contains a valid authenticator whose corresponding random-oracle input was never queried. Such an event is a direct tag forgery, bounded by the hash length and number of active sends. A previously accepted authenticator is instead rejected by the replay cache and cannot create a new accepted instance. In G_3, the test key is replaced by an independent random value unless the adversary queried $h(X \| Y)$ for the tested session.

Producing that final query requires reconstructing the long-term masking path, testing CEAF responses against a recorded Q_1, obtaining an accepted live response, or forging a preceding authenticator. Password and biometric uncertainty is collected in p_{cred}; the union of offline and live CEAF tests contributes $q_{\text{CEAF}}/26^s$, with the live component bounded only by the deployment's source/global admission policy. Define $\varepsilon_{\text{cred}} = \min\{1, 2p_{\text{cred}}(q_g)\}$ and $\varepsilon_{\text{CEAF}} = \min\{1, 2q_{\text{CEAF}}/26^s\}$. Applying the triangle inequality across the games gives

$$\text{Adv}_{\Pi}^{\text{ror}}(\text{A}) \le \min\left\{1, \frac{q_h(q_h-1)}{2^{l_h}} + \frac{q_n(q_n-1)}{2^{l_n}} + \frac{2q_s}{2^{l_h}} + \varepsilon_{\text{cred}} + \varepsilon_{\text{CEAF}}\right\}. \tag{2}$$

The bound makes two limits explicit. First, the CEAF contributes only $s \log_2 26 = 14.10$ bits at the selected parameters; a recorded transcript permits offline enumeration, while source/global admission control applies only to live attempts. Second, overall security depends on password distribution and biometric min-entropy rather than on the hash length alone. The outer minimum enforces the defining ROR range Adv ≤ 1; once $q_{\text{CEAF}} \ge 26^s/2$, the CEAF term saturates at the trivial bound rather than exceeding it. The main protocol excludes post-session long-term compromise.

In the three-credential experiment, (SC, PW, BIO) are known and the tested CEAF state is not. For recorded transcripts, q_e offline response tests replace the password/biometric term; live attempts additionally contribute q_c, with $q_e + q_c \le q_h$. The resulting bound is

$$\text{Adv}_{\Pi}^{\text{3cred}}(\text{A}) \le \min\left\{1, \frac{q_h(q_h-1)}{2^{l_h}} + \frac{q_n(q_n-1)}{2^{l_n}} + \frac{2q_s}{2^{l_h}} + \varepsilon_{\text{CEAF}}\right\}. \tag{3}$$

This is a bounded enumeration-and-online-guessing statement, not a claim of offline resistance after exposure of the other three credentials.

To complement the symbolic ROR argument, a supplied HLPSL abstraction of the core M_1–M_4 exchange returned SAFE with both AVISPA OFMC and CL-AtSe under bounded sessions (OFMC: four visited nodes at depth three; CL-AtSe: eight analyzed and three reachable states, typed model). The abstraction targets message-level secrecy and authentication; offline CEAF enumeration, source/global rate control, two-record recovery, M_5–M_6 retry, TEE extraction, and human usability remain outside its scope.

5.2 Protocol-Level Security Audit

5.2.1 Resistance to First-Factor Recovery

Q_1 binds the fresh challenge, CEAF response, target sensor, pseudo-identity, nonce, and timestamp. The card does not store a CEAF verifier, but a recorded Q_1 can be tested offline after the card, password, and biometric are exposed; live attempts are still checked at EN and are subject only to deployment source/global admission limits. The response state advances only after confirmation, so a dropped attempt cannot move one side prematurely. As Section 3.1.3 notes, two consecutive observations recover the CEAF state, and the protocol therefore makes no offline-resistance claim under three-credential compromise.

5.2.2 Resistance to Sensor-Node Capture

Capturing SN_j reveals its stored masked credentials and, with an observed current PID_j, that node's TC_j; the adversary can then attack current or future sessions of the captured node until revocation, and the protocol claims no more. Because every sensor uses an independently sampled R_j and a separate TC_j, this does not reconstruct another sensor's credential, so A4 concerns containment of lateral compromise to uncaptured nodes.

5.2.3 Session-Key Independence and the Absence of Perfect Forward Secrecy

Fresh independent (X, Y) give session-key independence: before long-term compromise, keys from separate accepted sessions are independent even when transcripts are public. This is weaker than perfect forward secrecy. A recorded transcript together with later (R_i, TC_i) or the edge record containing R_{EN} and PK_i recovers both nonces and past keys, so the main protocol lacks PFS; it protects SK from a channel-only adversary, not from trusted EN. Thus A12 in Table 3 must not be read as an A13 claim.

6 Comparison and Analytical Evaluation

We compare security properties with Li and Wang [54], Li et al. [24], Far et al. [34], and Xu et al. [25]. Their three-tier architectures are sufficiently close for a qualified property comparison. Quantitative cross-protocol rankings are not reported because the papers use different operation sets, hardware platforms, point sizes, and storage-inclusion rules; reproducing them under one verified implementation is outside the present analytical study.

6.1 Security-Property Comparison

Table 3 reports A12–A15 separately: key independence, PFS, update resilience, and edge-node key opacity.

Entries follow the original attack analyses and update procedures rather than a simple count of claimed features. A1 requires a cognitively evaluable factor that is verified remotely, not merely a dynamic identifier. A12 denotes independence of fresh session keys, whereas A13 requires resistance to later long-term compromise. A15 asks whether the authenticating intermediary can derive the endpoint key.

Far et al.'s PFS entry assumes erasure of its ephemeral value; its update is not recoverably synchronized and its gateway derives the key. Xu et al. provide A13–A15 under their ECC construction. Our main protocol deliberately lacks A13 and A15. Its A2/A3 entries are qualified because a recorded Q_1 enables offline CEAF enumeration after three-credential exposure, while live attempts are subject to deployment source/global limits. Our A14 covers the normal M_1–M_4 exchange, authenticated sensor resynchronization, and the bounded authenticated retry of M_5–M_6; rate limiting does not justify marking A8 as full DDoS resistance.

Cui et al. [26] provide message authentication without a comparable session key; Shang et al. [27] use two parties without a user or smart card. They are therefore excluded from the three-party tables.

6.2 Analytical Overhead of the Proposed Scheme

We report computation, communication, and storage for one complete login and AKA session; registration, update, replay caches, and session locks are excluded.

Table 3: Security-property comparison.

Property	Li and Wang [54]	Li et al. [24]	Far et al. [34]	Xu et al. [25]	Ours
A1	N	N	N	N	Y
A2	Y	Y	Y	Y	P[a]
A3	Y	Y	Y	Y	P[b]
A4	Y	Y	Y	Y	P[c]
A5	N	Y	Y	Y	Y
A6	Y	Y	Y	Y	Y
A7	Y	Y	Y	Y	Y
A8	N	N	Y	Y	N
A9	Y	Y	Y	Y	P[d]
A10	Y	Y	Y	Y	P[e]
A11	Y	Y	Y	Y	P[f]
A12	Y	Y	Y	Y	Y
A13	Y	Y	Y[g]	Y	N
A14	Y	Y	N	Y	P[h]
A15	Y	Y	N	Y	N

Y denotes the property under the stated model; P denotes a partial or qualified property; N denotes that the property is not provided. A1: cognitively evaluable authentication credential verified remotely; A2: smart-card theft and offline guessing; A3: online guessing; A4: sensor capture; A5: three-factor authentication; A6: message integrity; A7: replay; A8: DDoS; A9: mutual authentication and impersonation; A10: anonymity; A11: tracking; A12: session-key independence; A13: PFS; A14: desynchronization; A15: opacity to the authenticating intermediary. [a] Card-only compromise requires joint guessing; card plus password or biometric permits offline guessing of the remaining credential. With (SC, PW, BIO) exposed, a recorded Q_1 also permits offline CEAF enumeration. [b] A single admitted CEAF guess succeeds with probability 26^{-s}; over q_c admitted online trials, the union-bound success probability is at most $\min\{1, q_c/26^s\}$. The corresponding ROR advantage contribution is $2\min\{1, q_c/26^s\}$, while the full reduction also includes offline tests q_e. [c] The captured node remains compromised; P denotes containment of lateral compromise to other nodes. [d] Attribution at U assumes an honest trusted relay. [e] Anonymity is from the public channel, not from EN. [f] P denotes passive unlinkability only; an active drop can link adjacent runs while the real identity remains hidden. [g] Follows the source paper's erasure assumption. [h] P covers M_1–M_4, authenticated sensor resynchronization, and the bounded authenticated M_5–M_6 retry, not arbitrary repeated loss.

6.2.1 Cost Model and Overhead

We neglect random generation, timestamps, XOR, display, and character shifts in the analytical count; challenge expansion has expected cost $2.000008T_H$, rounded to $2T_H$. Secrets, nonces, hashes, and identities are modeled as 160 bits and timestamps as 32 bits under the H_{160} field profile defined in Section 3.1. With illustrative unit costs $T_H = 0.0004$ ms and $T_F = 0.442$ ms, the main protocol is analytically estimated at $15T_H + T_F + 21T_H + 6T_H = 42(0.0004) + 0.442 = 0.4588$ ms. This 0.4588 ms value is an illustrative analytical estimate, not an implementation benchmark. We additionally ran the reference implementation in `benchmarks/benchmark_protocol.py` for 200,000 iterations on a Windows 10 host with an AMD Ryzen 7 7800X3D 8-core processor (16 logical CPUs), Python 3.14.2, and the OpenSSL-backed `hashlib` SHA-256 implementation. The median time of one full 64-byte SHA-256 operation was $T_H^{\text{host}} = 0.000425$ ms (p95 0.000454 ms); protocol-field truncation to 160 bits is applied after the hash operation. The reference 256-byte fuzzy reproduction was $T_F^{\text{host}} = 0.004376$ ms (p95 0.004569 ms). The resulting role medians were 0.011004 ms for LD (15 hashes plus reproduction), 0.008994 ms for EN (21 hashes), and 0.002625 ms for SN (6 hashes), whose sum is 0.022624 ms. This is a host-software measurement, not a constrained-board, energy, or network benchmark; the fuzzy-reproduction value is implementation-dependent because the manuscript does not mandate a specific error-correcting fuzzy extractor. As an interim constrained-device calibration, Ledwaba et al. measured full SHA-256 on a Cortex-M4 at 57.361 μs/hash, 114.615 mW, and 3.56 kB of RAM [56]. The reported 114.615 mW is power,

Table 4: Analytical, host-measured, and literature-calibrated overhead of the proposed exchange. Communication and static-state entries use the 160-bit field profile; the 0.4588 ms value is an illustrative analytical estimate, host timing is a reference software measurement, and the Cortex-M4 row is a SHA-only estimate rather than a protocol benchmark.

(a) Computation

Scheme	*U*	*EN*	*SN*	Analytical (ms)	Host median (ms)
Ours	$15T_H + T_F$	$21T_H$	$6T_H$	0.4588	0.0226

(b) Communication (bits)

Msg	Route	Accounting	Main
M_1	$LD \rightarrow EN$	$5(160) + 32$	832
M_2	$EN \rightarrow SN$	$3(160) + 32$	512
M_3	$SN \rightarrow EN$	$2(160) + 32$	352
M_4	$EN \rightarrow LD$	$2(160) + 32$	352
Total			2048

(c) Static cryptographic state (bits)

Scope	*SC*/*LD*	*EN*	*SN*	Total
Base case ($m = n = 1$)	800	1440	640	2880

(d) Literature-calibrated Cortex-M4 estimate (SHA-only)

Role	SHA-256 calls	Time / energy (ms / mJ)	RAM reference
LD	15	$0.860 + T_F$ / $0.099 + E_F$	3.56 kB
EN	21	1.205 / 0.138	3.56 kB
SN	6	0.344 / 0.039	3.56 kB

The estimates apply $E_H = P_H t_H$ to the full-SHA-256 time and power reported for a Cortex-M4 in [56]; the reported power is not treated as an energy value, and E_F is not estimated because the protocol leaves the fuzzy extractor implementation open.

not per-hash energy; therefore, we first compute $E_H^{\mathrm{M4}} = P_H t_H = 114.615\,\mathrm{mW} \times 57.361\,\mu\mathrm{s} = 6.574\,\mu\mathrm{J} = 0.006574\,\mathrm{mJ}$ per hash, and then multiply by the role-specific hash counts. This gives the SHA-only estimates in Table 4, with the *LD* fuzzy-reproduction time and energy left as T_F and E_F. These are literature-calibrated estimates rather than measurements of this protocol; they exclude fuzzy extraction, I/O, radio, scheduling, compiler, and board-specific effects. Communication is 2048 bits for a normal session and static state is 2880 bits for $m = n = 1$ under the 160-bit field profile; an edge domain with m users and n sensors stores $160 + 800m + 640n$ bits at *EN*. The exceptional resynchronization exchange is excluded from the normal-session communication count. These counts exclude link, transport, retransmission, certificate, replay-cache, lock, and rate-limit overheads.

6.3 Sensitivity of the Guardrail Parameters

Because the live attack surface is the online CEAF term, the deployment parameters that shape that term are reported explicitly. The two protocol-relevant quantities are the segment count s, which fixes the response space 26^s, and the admitted online attempt budget q_c of Section 5.1.1. The values below are computed from the expressions already derived and are not measurements; they describe the security budget, not an implementation.

Table 5 varies s at $\ell = 7$. The success probability of one admitted CEAF guess is 26^{-s}, and the number of guesses needed for an even chance of one success grows as $26^s \ln 2$. Raising s mainly increases user selections; transmitted size is unchanged and challenge expansion adds less than one hash block.

Table 5: Analytical sensitivity of the online CEAF term to the segment count ($\ell = 7$).

s	L	26^s	Bits	One-guess probability	Guesses for 50%	$\mathbb{E}$[blocks]	Provisional guidance
2	14	676	9.40	1.48×10^{-3}	4.7×10^{2}	1.065	Provisional; low burden; weaker protection
3	21	17,576	14.10	5.69×10^{-5}	1.2×10^{4}	2.000	Provisional; validate budget/workflow
4	28	456,976	18.80	2.19×10^{-6}	3.2×10^{5}	2.027	Provisional; stronger; training advised
5	35	11,881,376	23.50	8.42×10^{-8}	8.2×10^{6}	2.787	Provisional; high burden; trained only

The final column is provisional deployment guidance, not an empirical usability conclusion. The guess counts are $26^s \ln 2$ and assume independent admitted attempts; they are policy arithmetic, not measurements. The final value of s should be selected jointly with the admitted-attempt budget q_c, observation environment, and target workflow. Human error rates under larger s are not modeled and require the usability study identified in Section 7.

The budget q_c behaves differently, and this behavior is why the scheme is described as rate-controlled rather than unconditionally resistant. The online term $2q_c/26^s$ is informative only while $q_c \ll 26^s/2$; at $s = 3$ it reaches unity at roughly 8.8×10^3 admitted attempts, and at $s = 4$ at roughly 2.3×10^5. Since nothing in the primitive caps q_c over a credential lifetime, the guarantee rests on the deployment enforcing source/global limits and on credential update resetting the state. A deployment that logs but does not bound admitted attempts obtains no security from this term regardless of s, which is the practical form of the negative result recorded in Sections 3.1.3 and 5.

7 Conclusions

We addressed dynamic credentials, multi-credential authentication, and lightweight three-party AKA for edge-intelligent IIoT. CEAF converts a fresh public challenge and an evolving memorized vector into a response collected by LD and bound into Q_1. After the password, smart card, and biometric are exposed, a recorded Q_1 still permits enumeration of the 26^s responses; live attempts remain subject to source/global limits with per-attempt success 26^{-s}. The scheme therefore makes no offline-resistance claim under three-credential compromise. Its contribution is transcript binding, explicit attack accounting, and a version-bound two-record state that recovers one lost M_1–M_4 message and invokes authenticated sensor resynchronization after repeated sensor-side loss.

The ROR reduction makes credential quality, CEAF enumeration, collisions, and forgery terms explicit. The AVISPA reports return SAFE for the bounded core model; the main protocol nevertheless provides session-key independence but neither PFS nor key opacity from the trusted edge node. The illustrative model gives 0.4588 ms, the host reference implementation gives a 0.0226 ms sum of role medians, and the Cortex-M4 calculation in Table 4 is a SHA-only literature estimate; none is a complete constrained-device or cross-protocol ranking. Password and CEAF remain knowledge credentials, and two consecutive observations reveal the CEAF step [57]; the one-observation bound is environmental, not cryptographic. EN remains a trusted key-derivation anchor. The two-record invariant absorbs one lost M_1–M_4 message, authenticated resynchronization repairs repeated sensor-side loss, and bounded M_5–M_6 retry handles one lost update acknowledgement.

Three concrete validation and mitigation paths follow from these boundaries. First, we will port the reference implementation to an ESP32-class board using an mbedTLS SHA-256 implementation, fix the clock/compiler configuration, and report cycle counts, RAM/flash use, current, and energy for each role; the present Cortex-M4 calculation is an interim calibration, not a substitute for that experiment. Second, a small usability pilot will measure completion time and error rate for the randomized $s = 3$ display before any deployment claim is made; no human-error statistic is asserted here. Third, an optional secure-element-held or login-device-TEE-held, non-extractable user-side HMAC key could be bound into Q_1 to prevent a recorded transcript from being tested with only (SC, PW, BIO), but that changes the credential architecture and requires a new proof and usability assessment. At the deployment layer, a stateless challenge/SYN-cookie-like admission token, source and global token buckets, concurrency caps, and replay filtering can reduce resource-exhaustion pressure [53]; these mechanisms mitigate rather than eliminate distributed DoS.

Acknowledgement: Not applicable.

Funding Statement: This research was funded by the National Natural Science Foundation of China, grant numbers 62272389 and 62372069.

Author Contributions: Xun Zhang: Conceptualization, Methodology, Validation, Formal analysis, Writing—original draft preparation, Funding acquisition; Zhiying Mu: Methodology, Investigation; Wu Wei: Validation, Formal analysis, Writing—review and editing; Zhihui Mu: Validation, Formal analysis, Writing—review and editing; Dejun Mu: Conceptualization, Validation, Writing—review and editing, Supervision, Funding acquisition; Xin Liu: Conceptualization, Validation, Writing—review and editing, Supervision, Funding acquisition. All authors reviewed and approved the final version.

Availability of Data and Materials: The reproducibility package is supplied as Supplementary Files S1–S6 and mirrored at the public GitHub repository. The repository contains the benchmark script and results under `benchmarks/` and the HLPSL model, latest AVISPA report, and WSL instructions under `avispa/`. The benchmark routine uses a deterministic 256-byte sample, a 16-byte salt, a SHA-256 helper digest, and two HMAC-SHA-256 calls as a reproducible timing surrogate; it is not a mandated error-correcting fuzzy extractor. The supplied results are reproducible subject to platform differences.

Ethics Approval: Not applicable.

Conflicts of Interest: The authors declare no conflicts of interest.

References

1. Shi W, Cao J, Zhang Q, et al. Edge computing: vision and challenges. IEEE Internet Things J. 2016;3:637-46.
2. Ranaweera P, Jurcut AD, Liyanage M. Survey on multi-access edge computing security and privacy. IEEE Commun Surv Tutor. 2021;23:1078-124.
3. Srinivas J, Mishra D, Mukhopadhyay S, et al. Provably secure biometric based authentication and key agreement protocol for wireless sensor networks. J Ambient Intell Humaniz Comput. 2018;9:875-95.
4. Jabbari A, Mohasefi JB. Improvement of a user authentication scheme for wireless sensor networks based on internet of things security. Wirel Pers Commun. 2021;116:2565-91.
5. Wu F, Xu L, Kumari S, et al. A privacy-preserving and provable user authentication scheme for wireless sensor networks based on internet of things security. J Ambient Intell Humaniz Comput. 2017;8:101-16.
6. Adil M, Almaiah MA, Omar Alsayed A, et al. An intelligent hybrid mutual authentication scheme for industrial internet of thing networks. Comput Mater Contin. 2021;68:447-70.
7. Wang D, Hong S, Wang Q. Revisiting a multifactor authentication scheme in industrial IoT. Secur Commun Netw. 2021;2021:9995832.
8. Singh J, Gimekar A, Venkatesan S. An efficient lightweight authentication scheme for human-centered industrial internet of things. Int J Commun Syst. 2023;36:e4189.
9. Shuai M, Xiong L, Wang C, et al. A secure authentication scheme with forward secrecy for industrial internet of things using Rabin cryptosystem. Comput Commun. 2020;160:215-27.
10. Rangwani D, Sadhukhan D, Ray S, et al. A robust provable-secure privacy-preserving authentication protocol for industrial internet of things. Peer-to-Peer Netw Appl. 2021;14:1548-71.
11. Basharzad SN, Fazeli M. Knowledge based dynamic password. In: Proceedings of the IEEE 4th International Conference on Knowledge-Based Engineering and Innovation; 2017. p. 367-72.
12. Liu X, Zhang RS, Zhao MQ. A robust authentication scheme with dynamic password for wireless body area networks. Comput Netw. 2019;161:220-34.
13. Shakirov B, Kim H, Lee K, et al. Analysis on vulnerability of password entry using virtual onscreen keyboard. J Korea Inst Inf Secur Cryptol. 2016;26:857-69.

14. Khalid Z, Paul P, Chattopadhyay SP, Biswas AN. Secure authentication with dynamic password. In: Proceedings of the 7th IEEE Annual Information Technology, Electronics and Mobile Communication Conference; 2016. p. 1-7.
15. Hitaj B, Gasti P, Ateniese G, Perez-Cruz F. PassGAN: a deep learning approach for password guessing. In: Applied Cryptography and Network Security (ACNS 2019). vol. 11464 of Lecture Notes in Computer Science; 2019. p. 217-37.
16. Xu M, Yu J, Zhang X, Wang C, Zhang S, Wu H, et al. Improving real-world password guessing attacks via bi-directional transformers. In: Proceedings of the 32nd USENIX Security Symposium; 2023. p. 1001-18.
17. Tolosana R, Vera-Rodriguez R, Fierrez J, Morales A, Ortega-Garcia J. DeepFakes and beyond: a survey of face manipulation and fake detection. Inf Fusion. 2020;64:131-48.
18. Kong DK, Wang XZ, Zhong LX, et al. Dynamic password token based on SM3 algorithm. In: Proceedings of the 5th International Conference on Audio, Language and Image Processing; 2016. p. 180-4.
19. Yusuf SI, Boukar MM, Mukhtar A, Yusuf AD. User define time based change pattern dynamic password authentication scheme. In: Proceedings of the 14th International Conference on Electronics, Computer and Computation; 2018. p. 206-12.
20. Chen L, Kurniati T, Wu ZY, et al. Using dynamic passwords for the exchange and sharing of personal health records: a reliable user authentication scheme. J Internet Technol. 2020;21:1049-59.
21. Turkanović M, Hölbl M. An improved dynamic password-based user authentication scheme for hierarchical wireless sensor networks. Electron Electr Eng. 2013;19(6):109-16.
22. Amin R, Biswas GP. A secure light weight scheme for user authentication and key agreement in multi-gateway based wireless sensor networks. Ad Hoc Netw. 2016;36:58-80.
23. Li X, Niu J, Kumari S, et al. A three-factor anonymous authentication scheme for wireless sensor networks in internet of things environments. J Netw Comput Appl. 2018;103:194-204.
24. Li X, Niu J, Bhuiyan MZA, et al. A robust ECC-based provable secure authentication protocol with privacy preserving for industrial internet of things. IEEE Trans Ind Inform. 2018;14:3599-609.
25. Xu H, Hsu CF, Harn L, et al. Three-factor anonymous authentication and key agreement based on fuzzy biological extraction for industrial internet of things. IEEE Trans Serv Comput. 2023;16:3000-13.
26. Cui J, Wang F, Zhang Q, et al. An anonymous message authentication scheme for semi-trusted edge-enabled IIoT. IEEE Trans Ind Electron. 2021;68:12921-9.
27. Shang W, Wen X, Chen Z, et al. Lightweight authentication scheme for edge control systems in industrial internet of things. Front Inf Technol Electron Eng. 2024;25:1466-78.
28. Wong KHM, Zheng Y, Cao JN, et al. A dynamic user authentication scheme for wireless sensor networks. In: Proceedings of the IEEE International Conference on Sensor Networks, Ubiquitous, and Trustworthy Computing; 2006. p. 244-51.
29. Das ML. Two-factor user authentication in wireless sensor networks. IEEE Trans Wirel Commun. 2009;8:1086-90.
30. Paliwal S. Hash-based conditional privacy preserving authentication and key exchange protocol suitable for industrial internet of things. IEEE Access. 2019;7:136073-93.
31. Turkanović M, Brumen B, Hölbl M. A novel user authentication and key agreement scheme for heterogeneous ad hoc wireless sensor networks, based on the internet of things notion. Ad Hoc Netw. 2014;20:96-112.
32. Jiang Q, Ma JF, Wei FS, et al. An untraceable temporal-credential-based two-factor authentication scheme using ECC for wireless sensor networks. J Netw Comput Appl. 2016;76:37-48.
33. Liu X, Guo Z, Ma J, et al. A secure authentication scheme for wireless sensor networks based on DAC and Intel SGX. IEEE Internet Things J. 2022;9(5):3533-47.
34. Abdi Nasib Far H, Bayat M, Das AK, et al. LAPTAS: lightweight anonymous privacy-preserving three-factor authentication scheme for WSN-based IIoT. Wirel Netw. 2021;27:1389-412.
35. Mohamed N, Al-Jaroodi J. Security applications of edge intelligence. In: Proceedings of the 2023 IEEE World AI IoT Congress (AIIoT). Seattle, WA, USA; 2023. p. 508-12.
36. Xu Y, Mao Y, Li J, Chen X, Wu S. Edge server enhanced secure and privacy preserving federated learning. Comput Netw. 2024;249:110465.
37. Wang K, Dong J, Xu Y, Ji X, Sha L, Xiao F. READ: resource efficient authentication scheme for digital twin edge networks. Future Gener Comput Syst. 2025;163:107498.
38. Xiao Z, Chen Y, Li T, Tian S, Min G, Tang Z, et al. Toward Secure Task Offloading in Vehicular Edge Computing Networks. IEEE Transactions on Dependable and Secure Computing. 2026;23(5):9963-77.
39. Sun G, Liao D, Zhao D, Xu ZE, Yu H. Live Migration for Multiple Correlated Virtual Machines in Cloud-based Data Centers. IEEE Transactions on Services Computing. 2018;11(2):279-91.
40. Zhang Y, Feng S, Wang P, Tan Z, Luo X, Ji Y, et al. Learning Self-Growth Maps for Fast and Accurate Imbalanced Streaming Data Clustering. IEEE Transactions on Neural Networks and Learning Systems. 2025;36(9):16049-61.
41. Yue M, Cui H, Gao T, Wu Z, Zheng H. ADS-B Tree-Chain: A Tree-Chain Authentication Scheme for Protecting ADS-B Data Security. IEEE Transactions on Aerospace and Electronic Systems. 2026;62:4525-39.
42. Zhang L, Ye Q, Yang W, He L. BRPPFL: A Byzantine-Robust and Privacy-Preserving Framework for Cloud–Edge Federated Learning. Computer Networks. 2026;286:112498.
43. Zhou Z, Chen X, Li E, Zeng L, Luo K, Zhang J. Edge intelligence: paving the last mile of artificial intelligence with edge computing. Proc IEEE. 2019;107(8):1738-62.
44. Deng S, Zhao H, Fang W, Yin J, Dustdar S, Zomaya AY. Edge intelligence: the confluence of edge computing and artificial intelligence. IEEE Internet Things J. 2020;7(8):7457-69.
45. Tramèr F, Zhang F, Juels A, Reiter MK, Ristenpart T. Stealing machine learning models via prediction APIs. In: Proceedings of the 25th USENIX Security Symposium; 2016. p. 601-18.
46. Shokri R, Stronati M, Song C, Shmatikov V. Membership inference attacks against machine learning models. In: Proceedings of the IEEE Symposium on Security and Privacy; 2017. p. 3-18.
47. Papernot N, McDaniel P, Sinha A, Wellman MP. SoK: security and privacy in machine learning. In: Proceedings of the IEEE European Symposium on Security and Privacy; 2018. p. 399-414.
48. Biggio B, Roli F. Wild patterns: ten years after the rise of adversarial machine learning. Pattern Recognit. 2018;84:317-31.
49. Vassilev A, Oprea A, Fordyce A, et al. Adversarial machine learning: a taxonomy and terminology of attacks and mitigations. Gaithersburg, MD, USA: National Institute of Standards and Technology; 2025. NIST AI 100-2e2025.
50. Greshake K, Abdelnabi S, Mishra S, Endres C, Holz T, Fritz M. Not what you've signed up for: compromising real-world LLM-integrated applications with indirect prompt injection. In: Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security; 2023. p. 79-90.
51. Carlini N, Jagielski M, Choquette-Choo CA, et al. Poisoning web-scale training datasets is practical. In: Proceedings of the IEEE Symposium on Security and Privacy; 2024. p. 407-25.
52. Qowi Z, Hudallah N. Combining caesar cipher and hill cipher in the generating encryption key on the vigenere cipher algorithm. J Phys Conf Ser. 2021;1918:042009.
53. Mirkovic J, Martin J, Reiher P. A Taxonomy of DDoS Attacks and DDoS Defense Mechanisms. ACM SIGCOMM Computer Communication Review. 2004;34(2):39-53.
54. Li W, Wang P. Two-factor authentication in industrial internet-of-things: attacks, evaluation and new construction. Future Gener Comput Syst. 2019;101:694-708.
55. Wang D, Cheng H, Wang P, Huang X, Jian G. Zipf's law in passwords. IEEE Trans Inf Forensics Secur. 2017;12(11):2776-91.
56. Ledwaba LPI, Hancke GP, Venter HS, Isaac SJ. Performance Costs of Software Cryptography in Securing New-Generation Internet of Energy Endpoint Devices. IEEE Access. 2018;6:9303-23.
57. Blum M, Vempala S. The complexity of human computation via a concrete model with an application to passwords. Proc Natl Acad Sci USA. 2020;117:9208-15.